

Architecture Discussions - Persistence and Reporting Adapters

Purpose

Capture the architectural boundary that has emerged in the modern web-monitor implementation: domain and application behavior should remain independent of ActivityInfo wherever practical, with provider-specific persistence and reporting concerns isolated behind adapters.

ActivityInfo is infrastructure, not the product interface

For web-monitor, ActivityInfo is primarily the persistence layer and, later, an aggregate-reporting provider. It is not the user interface for the monitoring application.

The user-facing path remains:

Web UI -> application/query services -> persistence abstraction -> ActivityInfo

The Flask web interface should present state, changes, history, and reports without requiring users to navigate the ActivityInfo database itself. Even when an ActivityInfo report is embedded later, it should appear as a feature of web-monitor rather than as an invitation to use ActivityInfo as the application UI.

This boundary is intentional. ActivityInfo can provide useful structured storage, subforms, permissions, and reporting capabilities while the monitoring system retains ownership of crawling, normalization, diffing, lifecycle semantics, historical queries, and presentation.

Persistence adapter boundary

The implementation now keeps ActivityInfo-specific HTTP, schema, reference, and query details behind the persistence layer.

Higher-level code works in terms of application concepts such as:

- Monitored Site
- Crawl
- Snapshot
- Snapshot Item
- historical state and change transitions

The crawler and statF1 6t cmnomaot thkw keat hativityInfo caextors. T monitoring syfecycle sek wshe persistence layer an usloadTj T* (quin

- retrieve Crawl/Snapshot history needed by historical queries.

The crawler, normalization, diff logic, lifecycle semantics, historical-query semantics, and eventual Flask UI should remain largely unchanged.

This is not a claim that storage replacement would be zero-work. Some current interfaces are structural rather than formalized as a single universal Python Protocol, and a different backend may require small integration adjustments or more efficient

Local development uses an ignored .env file. Production will use the same environment-variable names, with non-secret configuration supplied by the deployment environment and secrets supplied through Google Cloud Secret Manager.

Similarly, the ActivityInfo database itself is created manually. Bootstrap creates and verifies web-monitor's schema inside the configured database, while normal runtime code only reads and writes application records.

This separation keeps environment selection and provider credentials outside domain/application logic and makes future adapter selection easier.

Consequences for the remaining stages

Stage 5 should build the Flask UI entirely on the application/query services rather than raw ActivityInfo calls.

Stage 6 should preserve the reporting split described above: application-level reporting data first, ActivityInfo-specific publishing second.

Before Stage 7 deployment, perform a deployment-readiness review covering Google Cloud project selection, billing attachment, App Engine Standard setup/region, IAM permissions, Secret Manager access, Cloud Scheduler/API requirements, production ActivityInfo configuration, and budget safeguards.

Stage 7 should then be deployment and scheduling work rather than an architectural rewrite.

Design principle

web-monitor owns the meaning of a monitored state, a crawl, a change, a historical query, and a report metric. External systems such as ActivityInfo should provide storage or rendering capabilities through narrow boundaries, not define the application's core semantics.